ASSIGNMENT 1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)
Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____
Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level) Mandatory Execution and GitHub Submission
• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.
• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link:
https://github.com/192524025simats-collab/DSA0116_192524025.git

How to Answer the Additional Questions
Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.
Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.
Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse

and compare the time
complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Namma Metro Smart-Card Fare System
A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least
Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared on a card, and the fare values are typed in several places.

The number of journeys processed may grow:
1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day)

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact.

Design Challenge:
The metro is adding a fourth zone, a weekend discount and monthly passes.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects 4 Marks

Bloom's Level: K5 – Evaluate
Analyse the fare problem from the object-oriented point of view.
Discuss:
• The objects in the problem (card, passenger, journey, zone, fare table) and the data and behaviour of each
• How the same problem would be written in a procedural style, and what changes in an object-oriented style
• The basic principles of OOP – encapsulation, abstraction, inheritance and polymorphism – and where each could appear in this system
• The rules every correct journey must satisfy (for example: balance never below Rs 50 after

a journey)
You are expected to justify each object and rule, not merely list names.
Alternative Concept and Complexity Analysis
If the fare system were written in a purely procedural style (global variables and free functions) instead of SmartCard, Journey and FareTable classes, what would the solution be? Analyse whether the time per journey or the memory per card changes (both are O(1)), and compare how much code must be changed when one fare rule changes.
Evidence Expected
A clear object-oriented analysis with objects, their data and behaviour, rules and stated assumptions.

ANS:

Object-Oriented Analysis

1. **Objects in the Metro Fare System**

The Namma Metro Smart-Card System consists of different objects that work together to calculate fares and process passenger journeys.
Card: It stores the card ID and available balance. Its behaviours include checking the balance, recharging the card and deducting the fare after a successful journey.
Passenger: It stores the passenger ID, name and passenger category. Its behaviour includes identifying the passenger type and determining whether the passenger is eligible for a concession.
Journey: It stores the journey ID, source zone, destination zone, journey time and calculated fare. Its behaviours include calculating the fare, checking journey conditions and completing the journey.
Zone: It represents the Inner, Middle and Outer zones. It helps determine the fare according to the farthest zone travelled.
FareTable: It stores the base fares, peak-hour surcharge and applicable concession details. Its behaviour includes providing the correct fare and applying the required fare adjustments.
These objects are necessary because each one represents a separate responsibility in the metro system. Keeping them separate makes the program easier to understand, maintain and expand.

2. **Procedural Approach and Object-Oriented Approach**

In procedural programming, the metro fare system would be implemented using separate functions such as calculateFare(), checkPeakHour(), applyConcession(), checkBalance() and deductFare(). The card balance, passenger details and journey information would be stored in separate variables and passed between functions.
In object-oriented programming, the data and related functions are combined into classes such as Card, Passenger, Journey and FareTable. Each class manages its own data and operations.
The procedural approach is suitable for a small program, but managing a large number of journeys and additional fare rules can become difficult. The object-oriented approach provides better data protection, reusability, organization and maintainability. Therefore, OOP is more suitable for the expanding metro fare system.

3. **Basic Principles of OOP**

Encapsulation: Encapsulation combines data and functions within a class and protects the data from direct access. For example, the balance in the Card class can be declared private and modified only through methods such as getBalance() and deductFare(). This prevents unauthorized balance changes.
Abstraction: Abstraction hides internal implementation details and displays only the necessary

operations. For example, a passenger only needs to tap the smart card, while the system internally calculates the fare, checks concessions and validates the balance.
Inheritance: Inheritance allows one class to acquire the properties and functions of another class. For example, Student and SeniorCitizen classes can inherit common passenger details from a Passenger base class. Each category can have its own concession rules.
Polymorphism: Polymorphism allows the same function to perform different operations depending on the object. For example, a virtual function named calculateConcession() can be overridden in Student and SeniorCitizen classes to calculate their respective concessions.

4. Rules Every Correct Journey Must Satisfy
5. The source and destination zones must be valid.
6. The base fare must be selected according to the farthest zone travelled: Inner – Rs. 10, Middle – Rs. 20 and Outer – Rs. 30.
7. A 20% surcharge must be applied during peak hours: 8:00 AM to before 10:00 AM and 5:00 PM to before 8:00 PM.
8. Eligible students and senior citizens must receive the applicable concession.
9. The journey must be allowed only if the card balance after deducting the final fare is at least Rs. 50.
10. The card balance must never become negative.
11. Fare calculations must be accurate and should avoid floating-point rounding errors.
12. The fare must be deducted only after all journey conditions are successfully validated.

These rules ensure that every transaction is accurate, valid and secure.

Alternative Concept and Complexity Analysis

If the fare system were written using a purely procedural approach with global variables and separate functions instead of classes such as Card, Journey and FareTable, the program would store card balance, passenger details and fare information separately. Functions would operate on these values.
For example:

```
void deductFare(long long &balance, long long fare)
{
if (balance - fare >= 5000)
{
balance -= fare;
}
}
```

Both procedural and object-oriented approaches can process one journey in O(1) time when the same operations are performed. The memory required for a card balance also remains O(1). However, the object-oriented approach provides better organization and data protection. In the procedural approach, changes to fare rules may require modifications in several functions, whereas in the object-oriented approach, the relevant class can be updated with fewer changes to the rest of the system.
Therefore, the object-oriented approach is preferred for this metro system because it is easier to maintain and extend as the number of passengers and fare requirements increase.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly 3 Marks

Bloom's Level: K5 – Evaluate
Define the fares, surcharge and limits so that a revision is made in one place only. Discuss:

• The five kinds of tokens in your program (keywords, identifiers, constants, operators, punctuators) with examples
• #define, const and constexpr constants, and why constexpr is preferred for the fare table
• enum and enum class for zones, and why enum class is safer
• Naming rules and conventions for identifiers

Alternative Concept and Complexity Analysis
If the fares were written with #define macros instead of constexpr constants and an enum class, what would change? Compare type checking and scope, and analyse the memory used (a macro occupies no storage; a constexpr value usually does not either) and whether the running time is affected.
Evidence Expected
A single, well-named set of constants with a justified choice between #define, const, constexpr and enum class.

ANS:

Tokens, Keywords, Identifiers and Constants

1. **Five Types of Tokens in C++**

Tokens are the smallest individual units of a C++ program. The five main types of tokens are:

1. Keywords: Keywords are reserved words that have predefined meanings in C++. They cannot be used as variable or function names. Examples: int, class, public, private, if, else and return.
2. Identifiers: Identifiers are user-defined names given to variables, functions and classes. Examples: cardBalance, calculateFare(), Passenger and FareTable.
3. Constants: Constants are fixed values that cannot be changed during program execution. Examples: 10, 20, 30, 20 and 5000.
4. Operators: Operators are symbols used to perform arithmetic, comparison and logical operations. Examples: +, -, *, /, ==, >= and &&.
5. Punctuators: Punctuators are symbols used to structure and separate statements in a program. Examples: { }, ( ), ; and ,.
6. Defining Fares, Surcharge and Limits Using Constants

Constants should be declared in one place so that any changes to fare values or limits can be made without modifying multiple sections of the program.
The following C++ declarations can be used:

```
#include <iostream>
using namespace std;
constexpr long long INNER_FARE_PAISE = 1000;
constexpr long long MIDDLE_FARE_PAISE = 2000;
constexpr long long OUTER_FARE_PAISE = 3000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
enum class Zone
{
Inner,
Middle,
Outer
};
```

These constants store the fares in paise to avoid floating-point rounding errors. The surcharge

is defined as 20%, and the minimum remaining balance is stored as 5000 paise (Rs. 50).
#define is a preprocessor directive used to define symbolic macros. It performs text substitution before compilation. It does not provide normal C++ type checking or scope rules.
const declares a variable whose value cannot be modified after initialization. It provides type checking and follows normal C++ scope rules.
constexpr declares a constant expression that can be evaluated at compile time when its initializer permits it. It is suitable for fixed fare values because it provides type safety and compile-time evaluation.
constexpr is preferred for the fare table because the values are fixed, known during compilation and should be maintained in one place.

2. enum and enum class for Zones

An enum is used to define a group of named values representing related choices.
Example:

```
enum Zone
{
Inner,
Middle,
Outer
};
```

However, ordinary enum values can be implicitly converted to integers and may cause naming conflicts.
enum class provides stronger type safety and keeps its values within its own scope.
Example:

```
enum class Zone
{
Inner,
Middle,
Outer
};
Zone destination = Zone::Middle;
```

Here, Zone::Middle clearly identifies the selected zone. It cannot be accidentally treated as an ordinary integer without an explicit conversion.
Therefore, enum class is safer and more suitable for representing metro zones.

3. Naming Rules and Conventions for Identifiers

Identifiers must follow certain naming rules in C++:

1. An identifier must begin with a letter or underscore, not a digit.
2. It can contain letters, digits and underscores.
3. Spaces and special characters are not allowed.
4. C++ keywords cannot be used as identifiers.
5. Identifiers are case-sensitive. For example, cardBalance and CardBalance are different names.
6. Meaningful names should be used to improve readability.
7. Constants should preferably use uppercase letters with underscores, such as MIN_BALANCE_PAISE.
8. Class names should use meaningful names such as SmartCard, Passenger and FareTable.

Alternative Concept and Complexity Analysis
If the fares were defined using #define macros instead of constexpr constants and enum class, the program could be written as follows:

```
#define INNER_FARE_PAISE 1000
#define MIDDLE_FARE_PAISE 2000
#define OUTER_FARE_PAISE 3000
#define PEAK_SURCHARGE_PERCENT 20
#define MIN_BALANCE_PAISE 5000
#define INNER 0
#define MIDDLE 1
#define OUTER 2
```

In this approach, the preprocessor replaces each macro name with its corresponding value before compilation. Macros do not have normal data types or scope-based access control. They also do not provide the same type checking as constexpr constants and enum class. With constexpr, the compiler checks the declared data types and provides better protection against accidental misuse. With enum class, zone values are strongly typed and must be accessed using their scope, such as Zone::Inner.
In terms of memory, a macro occupies no storage because it is replaced during preprocessing. A constexpr value also usually requires no separate storage when used as a compile-time constant, although storage may be required in certain situations.
The running time is generally unaffected because both approaches can be evaluated or substituted during compilation. Both have O(1) time for accessing a fixed fare value. The memory usage is also generally O(1) for the fixed set of values.
Therefore, constexpr and enum class are preferred over #define macros because they provide better type checking, scope control, readability and maintainability.

Evidence Expected
A single set of well-named constexpr constants is used to define the Inner, Middle and Outer fares, peak surcharge and minimum balance. The enum class Zone provides type-safe zone representation. This design ensures that fare changes can be made in one place and reduces errors caused by repeated values.

TASK 3 – Choose Data Types and Check Type Compatibility 4 Marks

Bloom's Level: K5 – Evaluate
Select a C++ data type for every value in the system.
Discuss:
• The type chosen (bool, char, int, long long, unsigned, float, double, auto) for fares, balances, times, zone numbers and journey counts, with sizeof on your system
• Why money should be stored in integer paise and not as double rupees
• Type compatibility rules, and what the compiler does with int, double and bool values in the same expression
• The ranges from <climits> and whether int is enough for a day's collection across the network
Alternative Concept and Complexity Analysis
If money were stored as double rupees instead of long long paise, what would the solution be? Analyse the time per addition (O(1) in both), the space per value (8 bytes in both), and measure how the rounding error grows when n fares are added.

Evidence Expected
A justified data-type table with sizes and ranges, and evidence of one precision or overflow problem with its resolution.

ANS:

1. Data Type Selection

The following data types are selected for the metro smart-card system. The sizes shown are typical for a 64-bit system using g++.

Value	Data Type	Size	Reason
Fare and balance	long long	8 bytes	Stores money accurately in paise
Peak-hour status	bool	1 byte	Stores true or false
Passenger category	char	1 byte	Stores a single character
Time	int	4 bytes	Stores time in minutes
Zone number	unsigned int	4 bytes	Stores non-negative zone numbers
Journey count	unsigned int	4 bytes	Stores journey counts
Percentage	float	4 bytes	Stores decimal percentages
Decimal calculations	double	8 bytes	Provides higher floating-point precision
Automatically inferred values	auto	Depends on inferred type	Determines the type automatically

2. Why Money Should Be Stored in Integer Paise

Money should be stored as integer paise using long long instead of double rupees because floating-point numbers may produce rounding errors during calculations.

Example:

```
long long fare = 1050;    // Rs. 10.50
long long balance = 50000; // Rs. 500.00
```

Integer paise provides accurate addition and subtraction without floating-point rounding errors.

3. Type Compatibility Rules

- When int and double are used together, the int value is converted to double.
- A bool value is converted to int when used in arithmetic (true = 1 and false = 0).
- When a double value is assigned to an int, the fractional part is removed.
- Assigning a value outside a data type's range may cause overflow or data loss.

4. Ranges from <climits>

Data Type	Minimum Value	Maximum Value
int	-2,147,483,648	2,147,483,647

unsigned int	0	4,294,967,295
long long	-9,223,372,036,854,775,808	9,223,372,036,854,775,807

At 3 million journeys per day, assuming the maximum fare is Rs. 36 per journey:
Total collection = 3,000,000 × 36 = Rs. 108,000,000.
In paise, this is 10,800,000,000, which exceeds the maximum value of a 32-bit int. Therefore, long long is required to store the total collection in paise.

5. Alternative Concept and Complexity Analysis
If money were stored as double rupees, floating-point rounding errors could occur when adding many fares.
Example:

```
#include <iostream>
using namespace std;
int main() {
   double total = 0.0;
   for (int i = 0; i < 1000000; i++) {
      total += 0.1;
   }
   cout.precision(15);
   cout << total;
   return 0;
}
```

Expected mathematical result: 100000.0

The actual result may be slightly different due to floating-point rounding.

Comparison	long long paise	double rupees
Size	8 bytes	8 bytes
Time per addition	O(1)	O(1)
Space per value	8 bytes	8 bytes
Precision	Exact integer values	Possible rounding errors

As the number of additions increases, small floating-point errors can accumulate. The recommended solution is to store money as integer paise using long long and convert it into rupees only when displaying the fare.
Result:
Suitable data types were selected for the metro smart-card system. Integer paise using long long was identified as the reliable method for avoiding monetary precision errors and handling large daily collections.

TASK 4 – Control Implicit Conversions 4 Marks

Bloom's Level: K5 – Evaluate

Identify every place in the program where C++ converts a value from one type to another without being told.
Discuss:
• Integer division and integer-to-double conversion
• Comparison of signed and unsigned values, and the compiler warning it produces
• Narrowing conversions, and how brace initialisation (int x{3.7};) prevents them
• static_cast and when an explicit cast is needed
Predict the Output
Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>
int main()
{
 int a = 7, b = 2;
 double c = a / b;
 double d = static_cast<double>(a) / b;
 std::cout << c << ' ' << d << '\n';
 char ch = 'A' + 2;
 std::cout << ch << ' ' << ch + 1 << '\n';
 bool flag = 5;
 std::cout << flag << ' ' << std::boolalpha << flag << '\n';
 unsigned int u = 1;
 int s = -1;
 std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

My predicted output (write before running):
Explain each line of the actual output, naming the conversion that took place (integer division, char to int promotion, int to bool, signed to unsigned).
Alternative Concept and Complexity Analysis
If every conversion in the program were written explicitly with static_cast (and brace initialisation were used everywhere), instead of relying on implicit conversions, what would the solution be? Analyse whether the running time or memory changes, and which errors are caught at compile time instead of at run time.
Evidence Expected
Identified conversion points with demonstrated surprises, and corrected code using brace initialisation or static_cast.

ANS:

1. Implicit Conversions in C++
Implicit conversion occurs when the compiler automatically changes a value from one data type to another.

- **Integer division:** a / b performs integer division because both operands are integers. Therefore, 7 / 2 gives 3.
- **Integer to double:** The integer result 3 is converted to double when assigned to c.
- **Char to int promotion:** In ch + 1, the character is converted to an integer before addition.
- **Int to bool:** Assigning 5 to bool flag converts it to true.

- **Signed to unsigned:** During comparison of s and u, the negative signed integer is converted to an unsigned integer.

2. Signed and Unsigned Comparison

Comparing signed and unsigned integers can produce unexpected results. In this program, s = -1 is converted to an unsigned value before comparison with u = 1. Therefore, s < u is false.
The compiler may produce a warning such as:
warning: comparison of integer expressions of different signedness

3. Narrowing Conversions and Brace Initialisation

Narrowing conversions may cause data loss when converting a larger or decimal value into a smaller type.
Example:

```
int x = 3.7;  // Fractional part is lost
int y{3.7};   // Compilation error: narrowing conversion
```

Brace initialisation prevents unintended narrowing conversions by reporting them as compilation errors.

4. Use of static_cast

static_cast explicitly converts a value into another data type.
Example:

```
double result = static_cast<double>(7) / 2;
```

Output: 3.5
It is useful when a specific conversion is required, such as converting an integer to a double before division.

5. My Predicted Output (Before Running)

3 3.5
C 68
1 true
s >= u

6. Explanation of Actual Output

Output	Explanation
3 3.5	7 / 2 performs integer division and gives 3. static_cast<double>(a) converts 7 to double, producing 3.5 after division.
C 68	'A' + 2 produces character C. In ch + 1, ch is promoted to integer 67, resulting in 68.
1 true	Integer 5 is converted to Boolean true. By default, bool is printed as 1; boolalpha prints true.
s >= u	The signed value -1 is converted to unsigned before comparison, making the condition s < u false.

Actual Output:
3 3.5
C 68
1 true
s >= u
My predicted output and actual output are identical.

7. Corrected Code Using Explicit Conversions

```
#include <iostream>
using namespace std;

int main()
{
   int a{7}, b{2};

   double c{static_cast<double>(a) / static_cast<double>(b)};
   double d{static_cast<double>(a) / static_cast<double>(b)};

   cout << c << ' ' << d << '\n';

   char ch{static_cast<char>('A' + 2)};
   cout << ch << ' ' << static_cast<int>(ch) + 1 << '\n';

   bool flag{5 != 0};
   cout << boolalpha << flag << ' ' << flag << '\n';

   unsigned int u{1};
   int s{-1};

   cout << (static_cast<long long>(s) <
         static_cast<long long>(u) ? "s < u" : "s >= u") << '\n';

   return 0;
}
```

8. Alternative Concept and Complexity Analysis

If all conversions were written explicitly using static_cast and brace initialisation, the program would be easier to understand and safer.

- Time complexity: O(1) for each conversion and arithmetic operation.
- Space complexity: No significant change; the same variables are stored.
- Compile-time errors: Brace initialisation catches narrowing conversions.

Result:

Implicit conversions, integer division, signed and unsigned comparisons, and narrowing conversions were analysed. Explicit type conversions and brace initialisation help prevent unintended conversion errors.

TASK 5 – Write Correct Expressions with the Right Precedence 4 Marks

Bloom's Level: K5 – Evaluate

Write the expressions for the peak check, the surcharge and the concession.

Discuss:

• Operator precedence and associativity in each expression, including the conditional operator ?: and compound assignment

• The difference between x++ and ++x, and why i = i++ is a mistake

• The result of / and % with negative operands

• Why (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct and hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent but harder to read

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>
```

```
int main()
{
 int x = 5, y = 3;
 std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';
 int z = x++ + ++y;
 std::cout << z << ' ' << x << ' ' << y << '\n';
 std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
 std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';
 int k = 0;
 for (int i = 0; i < 10; i += 3) k += i;
 std::cout << k << '\n';
}
```

My predicted output (write before running):

For every line, name the operators involved and the order in which precedence and associativity apply them.

Alternative Concept and Complexity Analysis

If the peak-hour check were done with a lookup table bool peak[24] indexed by the hour, instead of a combined && / || expression, what would the solution be? Analyse the time per check (O(1) in both, but fewer comparisons with the table) and the extra space of 24 bytes.

Evidence Expected

Correct, readable expressions with an operator-by-operator explanation.

ANS:

1. Correct C++ Expressions

Peak-hour check:

```
bool peak = (hour >= 8 && hour < 10) ||
        (hour >= 17 && hour < 20);
```

Surcharge:

```
if (peak)
   fare += surcharge;
```

Concession:

```
if (student)
   fare -= concession;
```

2. Operator Precedence and Associativity

- Multiplication (*), division (/) and modulus (%) have higher precedence than addition (+) and subtraction (-).
- Relational operators are evaluated before logical AND (&&) and logical OR (||).
- The conditional operator (?:) has lower precedence than arithmetic and comparison operators.
- Compound assignment operators (+=, -=, *=) assign the calculated result back to the variable.
- Operators with the same precedence are generally evaluated according to their associativity.

3. Post-increment and Pre-increment

- x++ uses the current value first and increments afterwards.
- ++x increments first and then uses the updated value.
- i = i++ should be avoided because it is confusing and error-prone. In C++17 and later, the assignment overwrites the incremented value, so the increment is effectively lost.

4. Integer Division and Modulus

C++ integer division truncates the fractional part towards zero.

```
10 / 3   // 3
10 % 3   // 1
-10 / 3  // -3
-10 % 3  // -1
```
The remainder takes the sign of the dividend.

5. Peak Check with and without Parentheses

With parentheses:
```
(hour >= 8 && hour < 10) ||
(hour >= 17 && hour < 20)
```
Without parentheses:
```
hour >= 8 && hour < 10 ||
hour >= 17 && hour < 20
```
Both expressions produce the same result because && has higher precedence than ||. Parentheses improve readability.

6. Program Output and Explanation

Expected output:
```
11 16
9 6 4
6 4
3 1 -3 -1
18
```
Explanation:

1. x + y * 2 gives 11 because multiplication is evaluated first. (x + y) * 2 gives 16 because parentheses are evaluated first.
2. x++ + ++y gives 9. Post-increment uses the old value of x, while pre-increment updates y first.
3. The conditional operator selects the larger value, giving 6. The modulus and multiplication expression gives 4.
4. Integer division and modulus produce 3, 1, -3, -1.
5. The loop calculates the final value of k as 18.

7. Alternative Peak-Hour Lookup Table
```
bool peak[24] = {false};
peak[8] = true;
peak[9] = true;
peak[17] = true;
peak[18] = true;
peak[19] = true;
bool isPeak = peak[hour];
```

Comparison:

Method	Time Complexity	Extra Space
Logical expression	O(1)	O(1)
Lookup table	O(1)	24 bytes

Result

Thus, operator precedence, associativity, increment operators, integer division, modulus and

lookup-table implementation were studied and evaluated using C++ expressions and program output.

TASK 6 – Design the Control Structures 4 Marks

Bloom's Level: K5 – Evaluate

Design the decision and looping logic for computing a fare and processing a day's journeys. Discuss:

• When to use if–else, a switch on the zone, and the conditional operator

• How a for loop, a range-based for loop and a while loop could process the journeys, and which is best

• Where break and continue are needed (for example, skipping an invalid journey) • What happens when no case of a switch matches

Write pseudocode for the fare computation following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing C++ code.

Complete the Code

Complete TODO 1 and TODO 2 below using a switch statement and integer paise, and execute the program.

```
#include <iomanip>
#include <iostream>
enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs
20, Rs 30 constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must remain on the card
long long fare(Zone z, bool peak)
{
 // TODO 1: Use a switch on z to pick the base fare from BASE_PAISE,
then add  // PEAK_SURCHARGE_PERCENT % in peak hours using
integer arithmetic only
}
bool canTravel(long long balance, long long f)
{
 // TODO 2: Return true only if the balance left after paying the fare is at
least  // MIN_BALANCE_PAISE

}
int main()
{
 long long balance = 10000; // Rs 100.00 in paise
 struct Trip { Zone zone; bool peak; };
 Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true},
{Zone::Middle, false}};  for (const Trip& t : trips) {
 long long f = fare(t.zone, t.peak);
 if (canTravel(balance, f)) {
 balance -= f;
 std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
 << std::setfill('0') << f % 100 << std::setfill(' ');
 } else {
 std::cout << "Denied (low balance)";
```

```
 }
 std::cout << " balance Rs " << balance / 100 << '\n';
 }
}
```

Expected output:

Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54

Add a journey at exactly 10 AM and at exactly 8 PM and check that your peak rule treats them correctly.

Alternative Concept and Complexity Analysis

If the base fare were looked up from an array indexed by the zone (constexpr long long FARE[] = {1000, 2000, 3000}) instead of a switch, what would the solution be? Analyse the time per lookup (O(1) in both; the switch may become a jump table), the space of the array, and how each design copes with a fourth zone.

Evidence Expected

Pseudocode and C++ control structures for the fare computation, with justified choices.

ANS:

1. Selection Control Structures

if–else statement:

Used to check conditions such as whether the passenger has sufficient balance to travel.

```
if (canTravel(balance, fare))
{
   balance -= fare;
}
else
{
   cout << "Denied (low balance)";
}
```

switch statement:

Used to select the base fare according to the passenger's travel zone (Inner, Middle or Outer).

Conditional operator:

Used for simple two-way decisions. It is a shorter alternative to an if–else statement.

2. Looping Structures

- for loop: Used when the number of journeys is known.
- Range-based for loop: Used to process each journey in an array or collection.
- while loop: Used when the number of journeys is not known and depends on a condition.

Best choice: A range-based for loop is best for processing the given journey array because it directly accesses each journey without requiring an index.

3. Use of break and continue

- break terminates a loop or switch statement.
- continue skips the current iteration and moves to the next iteration.
- continue can be used to skip an invalid journey without processing its fare.

4. Switch Statement Without a Matching Case

If no case matches the given value, the default block executes. If no default block is provided, the switch statement is skipped.

5. Algorithmic Pseudocode

Algorithm: Calculate Fare and Process Journeys
Input: Zone, peak-hour status and available balance.
Output: Fare amount and remaining balance.

```
BEGIN

   SET balance = 10000

   FOR each journey in trips DO

      SELECT journey.zone

         CASE Inner:
            SET baseFare = 1000

         CASE Middle:
            SET baseFare = 2000

         CASE Outer:
            SET baseFare = 3000

         DEFAULT:
            CONTINUE

      END SELECT

      IF journey.peak = TRUE THEN
         SET fare = baseFare + (baseFare * 20 / 100)
      ELSE
         SET fare = baseFare
      END IF

      IF balance - fare >= 5000 THEN
         SET balance = balance - fare
         DISPLAY fare and balance
      ELSE
         DISPLAY "Denied (low balance)"
         DISPLAY balance
      END IF

   END FOR
END
```

6. Complete C++ Program

```
TODO 1 – Fare calculation using switch statement
TODO 2 – Balance validation
#include <iomanip>
#include <iostream>
using namespace std;

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;

constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak)
{
```

```
    long long baseFare = 0;

    switch (z)
    {
        case Zone::Inner:
            baseFare = BASE_PAISE[0];
            break;

        case Zone::Middle:
            baseFare = BASE_PAISE[1];
            break;

        case Zone::Outer:
            baseFare = BASE_PAISE[2];
            break;

        default:
            return 0;
    }

    if (peak)
    {
        baseFare += baseFare * PEAK_SURCHARGE_PERCENT / 100;
    }

    return baseFare;
}

bool canTravel(long long balance, long long f)
{
    return (balance - f >= MIN_BALANCE_PAISE);
}

int main()
{
    long long balance = 10000;

    struct Trip {
        Zone zone;
        bool peak;
    };

    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };

    for (const Trip& t : trips)
    {
        long long f = fare(t.zone, t.peak);

        if (canTravel(balance, f))
        {
            balance -= f;

            cout << "Fare Rs " << f / 100 << '.'
                 << setw(2) << setfill('0') << f % 100
                 << setfill(' ');
        }
        else
```

```
        {
            cout << "Denied (low balance)";
        }

        cout << " balance Rs " << balance / 100 << '\n';
    }

    return 0;
}
```

7. Expected Output

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

8. Peak-Hour Boundary Testing

Peak hours are 8 AM to before 10 AM and 5 PM to before 8 PM.

The condition is:

```
bool peak = (hour >= 8 && hour < 10) ||
            (hour >= 17 && hour < 20);
```

Time	Peak Status
8:00 AM	Peak
10:00 AM	Non-peak
5:00 PM	Peak
8:00 PM	Non-peak

Therefore, exactly 10 AM and exactly 8 PM are treated as non-peak hours.

9. Alternative Array-Based Approach

Instead of using a switch statement, the base fare can be selected using an array indexed by the zone.

```
constexpr long long FARE[] = {1000, 2000, 3000};

long long fare(Zone z, bool peak)
{
    long long baseFare = FARE[static_cast<int>(z)];

    if (peak)
        baseFare += baseFare * 20 / 100;

    return baseFare;
}
```

Complexity Analysis:

Feature	Switch Statement	Array Lookup
Time complexity	O(1)	O(1)
Extra space	O(1)	O(1)

Selection	Based on matching cases	Based on index
Fourth zone	Add another case	Add another array element

The array requires 24 bytes when using three long long elements (assuming 8 bytes each). A fourth zone requires adding another element. In the switch approach, a fourth case must be added.

10. Result

Thus, fare computation and journey processing were implemented using switch statements, if–else conditions and range-based for loops. Integer paise arithmetic was used to calculate fares accurately, and balance validation ensured that at least Rs. 50 remained on the travel card. Peak-hour boundary conditions and alternative array-based fare selection were also evaluated.

TASK 7 – From Procedural to Object-Oriented 4 Marks

Bloom's Level: K5 – Evaluate

Rewrite the fare logic in two styles.

Discuss:

• A procedural version with separate variables and free functions

• An object-oriented version with a SmartCard class that keeps its balance private and offers deduct() and canTravel()

• How the class prevents a negative balance

• Which version is easier to change when the minimum balance changes

Alternative Concept and Complexity Analysis

If the balance were kept in a public data member instead of a private member with deduct() and canTravel(), what would the solution be? Analyse whether time or space changes (they do not), and explain which rule of the system can then be broken by any part of the program.

Evidence Expected

Two working versions and a reasoned comparison of the procedural and object-oriented designs.

ANS:

1. Procedural Programming Approach

In procedural programming, separate variables and functions are used to calculate fares and manage the travel card balance.

C++ Program:

```
#include <iostream>
using namespace std;

long long balance = 10000;
const long long MIN_BALANCE = 5000;

bool canTravel(long long fare)
{
   return balance - fare >= MIN_BALANCE;
}
```

```
void deduct(long long fare)
{
    if (canTravel(fare))
    {
        balance -= fare;
        cout << "Fare deducted: Rs " << fare / 100.0 << endl;
    }
    else
    {
        cout << "Travel denied (low balance)" << endl;
    }
}

int main()
{
    deduct(1000);
    deduct(3000);
    deduct(2000);

    cout << "Remaining balance: Rs "
         << balance / 100.0 << endl;

    return 0;
}
```

Output:

Fare deducted: Rs 10

Fare deducted: Rs 30

Travel denied (low balance)

Remaining balance: Rs 60

2. Object-Oriented Programming Approach

In object-oriented programming, a SmartCard class combines balance data and related functions. The balance is kept private and accessed through public member functions.

C++ Program:

```
#include <iostream>
using namespace std;

class SmartCard
{
private:
    long long balance;
    const long long MIN_BALANCE = 5000;

public:
    SmartCard(long long initialBalance)
    {
        balance = initialBalance;
    }

    bool canTravel(long long fare)
    {
        return balance - fare >= MIN_BALANCE;
    }

    void deduct(long long fare)
    {
        if (canTravel(fare))
        {
            balance -= fare;
```

```
            cout << "Fare deducted: Rs "
                 << fare / 100.0 << endl;
        }
        else
        {
            cout << "Travel denied (low balance)" << endl;
        }
    }

    long long getBalance()
    {
        return balance;
    }
};

int main()
{
    SmartCard card(10000);

    card.deduct(1000);
    card.deduct(3000);
    card.deduct(2000);

    cout << "Remaining balance: Rs "
         << card.getBalance() / 100.0 << endl;

    return 0;
}
```

Output:

Fare deducted: Rs 10
Fare deducted: Rs 30
Travel denied (low balance)
Remaining balance: Rs 60

3. How the Class Prevents a Negative Balance

- The balance is declared as a private data member.
- Other parts of the program cannot directly modify it.
- The canTravel() function checks whether sufficient balance remains.
- The deduct() function deducts the fare only when the condition is satisfied.

This protects the balance from invalid deductions.

4. Which Version Is Easier to Change?

The object-oriented version is easier to maintain because the minimum balance rule is stored inside the SmartCard class. If the minimum balance changes, it can be updated in one place without modifying multiple functions.

5. Alternative: Public Balance Member

If the balance were public, any part of the program could directly change it.

Example:

```
class SmartCard
{
public:
    long long balance = 10000;
};

int main()
{
    SmartCard card;

    card.balance = -5000;
```

```
    cout << "Balance: " << card.balance;

    return 0;
}
```

This allows an invalid negative balance.

Complexity Analysis:

Feature	Private Balance	Public Balance
Time complexity	O(1)	O(1)
Space complexity	O(1)	O(1)
Data protection	Protected	Not protected
Balance validation	Controlled through methods	Can be bypassed

A public balance allows any part of the program to break the system rule that the card must maintain at least Rs. 50 after a journey.

6. Comparison of Procedural and Object-Oriented Programming

Procedural Programming	Object-Oriented Programming
Uses separate variables and functions	Uses classes and objects
Data may be accessed directly	Data can be hidden using private
Less data protection	Better data protection
Changes may affect multiple functions	Changes can be managed within the class
Suitable for simple programs	Suitable for larger applications

Result

Thus, the fare management logic was implemented using both procedural and object-oriented approaches. The SmartCard class provides better data security through encapsulation and prevents invalid balance deductions. Therefore, object-oriented programming is more suitable for maintaining the smart card fare system.

TASK 8 – Read and Print Data Safely 4 Marks

Bloom's Level: K5 – Evaluate

Design the input and output of the gate program.

Discuss:

• Reading values with cin and checking cin.fail() when the operator types letters instead of numbers

• Clearing the error state with cin.clear() and discarding the bad input

• Formatting money with <iomanip> (setw, setfill, fixed, setprecision)

• Printing a neat journey report

Alternative Concept and Complexity Analysis

If each input line were read with std::getline() and converted with std::stoi() inside try/catch,

instead of reading numbers directly with cin >>, what would the solution be? Analyse the time to process a line of length L (O(L) in both) and the buffer space needed, and compare how each recovers from invalid input.
Evidence Expected
Safe input handling with demonstrated recovery from invalid input, and correctly formatted output.

ANS:

1. Reading Input Using cin
The cin statement is used to read input from the user. The cin.fail() function checks whether the input operation has failed, such as when the user enters letters instead of numbers.
Example:

```
int age;

cout << "Enter age: ";
cin >> age;

if (cin.fail())
{
   cout << "Invalid input!";
}
```

2. Clearing Invalid Input

- cin.clear() clears the error state of the input stream.
- cin.ignore() discards the remaining invalid input.
- These functions allow the program to accept input again.

Example:

```
#include <iostream>
#include <limits>
using namespace std;

int readNumber()
{
   int value;

   while (true)
   {
      cout << "Enter a number: ";
      cin >> value;

      if (!cin.fail())
         return value;

      cout << "Invalid input. Try again.\n";

      cin.clear();
      cin.ignore(numeric_limits<streamsize>::max(), '\n');
   }
}
```

3. Formatting Money Using iomanip
The <iomanip> header provides formatting functions to display money neatly.

- setw() sets the width of the output.
- setfill() fills empty spaces with a specified character.
- fixed displays floating-point values in fixed notation.
- setprecision(2) displays two digits after the decimal point.

Example:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    double fare = 36.5;

    cout << "Fare: Rs "
         << fixed << setprecision(2)
         << fare << endl;

    return 0;
}
```

Output:

```
Fare: Rs 36.50
```

4. Complete C++ Program – Safe Input and Journey Report

```
#include <iostream>
#include <iomanip>
#include <limits>
using namespace std;

int readNumber(string message)
{
    int value;

    while (true)
    {
        cout << message;
        cin >> value;

        if (!cin.fail())
            return value;

        cout << "Invalid input! Enter numbers only.\n";

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

int main()
{
    int journeys;
    double fare, total = 0;

    journeys = readNumber("Enter number of journeys: ");

    cout << "\n--- Journey Report ---\n";

    for (int i = 1; i <= journeys; i++)
    {
        fare = readNumber("Enter fare for journey: ");

        if (fare < 0)
        {
            cout << "Invalid fare. Enter again.\n";
            i--;
```

```
            continue;
        }

        total += fare;

        cout << "Journey " << i
             << " Fare: Rs "
             << fixed << setprecision(2)
             << fare << endl;
    }

    cout << "\nTotal Fare: Rs "
         << fixed << setprecision(2)
         << total << endl;

    return 0;
}
```

5. Sample Output

```
Enter number of journeys: 2

--- Journey Report ---
Enter fare for journey: abc
Invalid input! Enter numbers only.
Enter fare for journey: 10
Journey 1 Fare: Rs 10.00
Enter fare for journey: 36
Journey 2 Fare: Rs 36.00

Total Fare: Rs 46.00
```

6. Alternative Approach – getline() and stoi()

Instead of using cin >>, the program can read a complete input line using getline() and convert it into an integer using stoi().
The try-catch block handles invalid input safely.

Example:

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string input;
    int number;

    cout << "Enter a number: ";
    getline(cin, input);

    try
    {
        number = stoi(input);
        cout << "Entered number: " << number;
    }
    catch (const exception&)
    {
        cout << "Invalid input!";
    }

    return 0;
}
```

7. Complexity Analysis

Feature	cin with fail()	getline() with stoi()
Time complexity	O(L)	O(L)
Buffer space	Input stream buffer	String buffer of O(L)
Invalid input handling	cin.clear() and cin.ignore()	try-catch
Input method	Reads formatted values	Reads a complete line
Error recovery	Clears stream error state	Handles conversion exceptions

Here, L represents the length of the input line.
Both approaches require O(L) time to process an input line. The getline() approach stores the complete line in a string, while cin uses the input stream buffer.

Result

Thus, safe input handling and formatted output were implemented using cin, cin.fail(), cin.clear(), cin.ignore() and <iomanip>. An alternative input method using getline() and stoi() was also demonstrated and compared based on time complexity, memory usage and error recovery.

TASK 9 – Implement and Test the Fare Logic 4 Marks

Bloom's Level: K5 – Evaluate
Implement the complete fare logic and test it.
Test:
• Normal journeys in each zone, peak and off-peak
• Boundary cases: exactly 8:00, 10:00, 17:00 and 20:00; a balance that leaves exactly Rs 50 • Invalid cases: an unknown zone, a negative recharge, letters typed for the hour • A small set of 5 journeys whose total you calculate by hand
Alternative Concept and Complexity Analysis
If each journey were tested against a table of test cases (inputs and expected fares) in a loop, instead of separate hand-written checks, what would the solution be? Analyse the time for t tests (O(t)) and the space of the table, and how easily a new boundary case can be added.
Evidence Expected
Working source code and a test table with expected and actual results for every case.

ANS:

1. Objective

To implement the complete smart card fare calculation system and test it using normal journeys, peak and off-peak hours, boundary conditions and invalid inputs.

2. Fare Calculation Rules

Zone	Base Fare
Inner	Rs. 10
Middle	Rs. 20

Outer	Rs. 30

- Peak surcharge: 20%
- Peak hours: 8:00 AM–before 10:00 AM and 5:00 PM–before 8:00 PM.
- Minimum remaining balance: Rs. 50.
- All fare calculations are performed using integer paise.

3. Complete C++ Program

```
#include <iostream>
#include <iomanip>
#include <string>
#include <limits>
using namespace std;

enum class Zone { Inner, Middle, Outer };

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

bool isPeak(int hour)
{
   return (hour >= 8 && hour < 10) ||
         (hour >= 17 && hour < 20);
}

bool getHour(int &hour)
{
   cout << "Enter hour (0-23): ";

   if (!(cin >> hour))
   {
      cout << "Invalid hour input!\n";
      cin.clear();
      cin.ignore(numeric_limits<streamsize>::max(), '\n');
      return false;
   }

   if (hour < 0 || hour > 23)
   {
      cout << "Hour must be between 0 and 23.\n";
      return false;
   }

   return true;
}

bool getZone(int choice, Zone &z)
{
   switch (choice)
   {
      case 1:
         z = Zone::Inner;
         return true;

      case 2:
         z = Zone::Middle;
         return true;
```

```
        case 3:
            z = Zone::Outer;
            return true;

        default:
            return false;
    }
}

long long calculateFare(Zone z, bool peak)
{
    long long f = 0;

    switch (z)
    {
        case Zone::Inner:
            f = BASE_PAISE[0];
            break;

        case Zone::Middle:
            f = BASE_PAISE[1];
            break;

        case Zone::Outer:
            f = BASE_PAISE[2];
            break;
    }

    if (peak)
        f += f * PEAK_SURCHARGE_PERCENT / 100;

    return f;
}

bool canTravel(long long balance, long long f)
{
    return balance >= f &&
           balance - f >= MIN_BALANCE_PAISE;
}

int main()
{
    long long balance = 10000;
    int choice, hour;

    cout << "SMART CARD FARE SYSTEM\n";
    cout << "1. Inner Zone\n";
    cout << "2. Middle Zone\n";
    cout << "3. Outer Zone\n";

    cout << "Enter zone: ";
    if (!(cin >> choice))
    {
        cout << "Invalid zone input!\n";
        return 0;
    }

    Zone z;

    if (!getZone(choice, z))
```

```
    {
        cout << "Unknown zone!\n";
        return 0;
    }

    if (!getHour(hour))
        return 0;

    bool peak = isPeak(hour);
    long long f = calculateFare(z, peak);

    cout << fixed << setprecision(2);

    if (canTravel(balance, f))
    {
        balance -= f;

        cout << "Fare: Rs. "
             << f / 100.0 << '\n';
    }
    else
    {
        cout << "Travel denied (low balance)\n";
    }

    cout << "Remaining balance: Rs. "
         << balance / 100.0 << '\n';

    return 0;
}
```

4. Test Cases and Expected Results

Test	Zone	Hour	Expected Fare
1	Inner	11	Rs. 10
2	Inner	8	Rs. 12
3	Middle	11	Rs. 20
4	Middle	17	Rs. 24
5	Outer	11	Rs. 30
6	Outer	19	Rs. 36
7	Inner	8	Rs. 12
8	Inner	10	Rs. 10
9	Inner	17	Rs. 12
10	Inner	20	Rs. 10

Boundary observations:

- 8:00 AM – Peak hour.
- 10:00 AM – Non-peak hour.

- 5:00 PM – Peak hour.
- 8:00 PM – Non-peak hour.

5. Invalid Input Testing

Test Case	Input	Expected Result
Unknown zone	5	Unknown zone
Negative recharge	-500	Recharge rejected
Letters for hour	abc	Invalid hour input
Hour outside range	25	Invalid hour

```
long long recharge(long long amount)
{
   if (amount < 0)
   {
      cout << "Invalid recharge amount!\n";
      return 0;
   }

   return amount;
}
```

6. Balance Boundary Testing

Initial balance = Rs. 60
Fare = Rs. 10
Remaining balance = Rs. 50.
Expected result: Travel is allowed because the remaining balance is exactly Rs. 50.

7. Five-Journey Manual Calculation

Initial balance = Rs. 200.

Journey	Zone	Time	Fare
1	Inner	11 AM	Rs. 10
2	Middle	8 AM	Rs. 24
3	Outer	11 AM	Rs. 30
4	Inner	5 PM	Rs. 12
5	Middle	1 PM	Rs. 20
Total			Rs. 96

Remaining balance:
Rs. 200 − Rs. 96 = Rs. 104
All five journeys are accepted because the remaining balance stays above Rs. 50.

8. Alternative Approach – Table-Driven Testing

```
struct TestCase
{
```

```
    Zone zone;
    int hour;
    long long expectedFare;
};

TestCase tests[] =
{
    {Zone::Inner, 11, 1000},
    {Zone::Inner, 8, 1200},
    {Zone::Middle, 11, 2000},
    {Zone::Middle, 17, 2400},
    {Zone::Outer, 11, 3000},
    {Zone::Outer, 19, 3600},
    {Zone::Inner, 10, 1000},
    {Zone::Inner, 20, 1000}
};

for (const TestCase& t : tests)
{
    long long actual = calculateFare(t.zone, isPeak(t.hour));

    cout << "Expected: " << t.expectedFare
         << " Actual: " << actual
         << (actual == t.expectedFare ? " PASS" : " FAIL")
         << '\n';
}
```

9. Complexity Analysis

Feature	Analysis
Time for one test	O(1)
Time for t tests	O(t)
Space for test table	O(t)
Adding a boundary case	Easy

10. Result

Thus, the smart card fare calculation system was implemented and evaluated using C++. Normal journeys, peak and off-peak hours, boundary conditions, invalid inputs and balance restrictions were tested. Table-driven testing was also demonstrated to improve test organization and simplify the addition of new test cases.

TASK 10 – Compare Expectations with Execution 4 Marks

Bloom's Level: K5 – Evaluate

Compare what you expected with what actually happened.

Discuss:

• Whether the double and integer-paise totals for 20,000, 500,000 and 3,000,000 journeys agree, and by how much they differ

• Whether int overflows for the network total

• The compiler warnings you received with -Wall -Wextra and what each meant • Reasons for any difference

Alternative Concept and Complexity Analysis
If the network total were accumulated in an int, a long long or a double, what would happen at 3,000,000 journeys? Analyse when each type fails (overflow versus loss of precision), and compare the time per addition and the space per variable.
Evidence Expected
A reasoned comparison of expected and observed behaviour, supported by measured totals.

ANS:

1. Objective

To compare floating-point and integer-paise fare totals for 20,000, 500,000 and 3,000,000 journeys and evaluate integer overflow, precision and compiler warnings.

2. Expected vs Actual Totals

Number of Journeys	Double Total (Rs.)	Integer-Paise Total	Total in Rs.
20,000	600,000	60,000,000 paise	600,000
500,000	15,000,000	1,500,000,000 paise	15,000,000
3,000,000	90,000,000	9,000,000,000 paise	90,000,000

Observation:

- Both methods produce the same totals for these whole-rupee fares.
- The difference between the double and integer-paise totals is Rs. 0.
- Integer paise avoids floating-point rounding errors when representing money.

3. Integer Overflow

A normal int generally has a maximum value of 2,147,483,647.
At 3,000,000 journeys:

$$3,000,000 \times 3000 = 9,000,000,000$$

This exceeds the maximum value of a 32-bit int.
Therefore, integer overflow occurs if the network total is stored in an int.
A long long can safely store this total.

4. C++ Program for Comparison

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
   const long long farePaise = 3000;

   int journeys[] = {20000, 500000, 3000000};

   for (int n : journeys)
   {
      double doubleTotal = 0.0;
      long long integerTotal = 0;

      for (int i = 0; i < n; i++)
      {
```

```
            doubleTotal += 30.00;
            integerTotal += farePaise;
        }

        cout << "Journeys: " << n << endl;

        cout << fixed << setprecision(2);

        cout << "Double total: Rs. "
             << doubleTotal << endl;

        cout << "Integer-paise total: Rs. "
             << integerTotal / 100.0 << endl;

        cout << "Difference: Rs. "
             << doubleTotal - integerTotal / 100.0
             << "\n\n";
    }

    return 0;
}
```

5. Compiler Warnings

The program can be compiled using:

```
g++ -Wall -Wextra task10.cpp -o task10
```

-Wall enables many common compiler warnings.

- -Wextra enables additional warnings.
- These options help identify unused variables, suspicious conditions and other potential programming mistakes.

The given comparison program should not produce ordinary warnings with these options. However, signed integer overflow is not guaranteed to produce a compiler warning.

6. Reasons for Differences

- Floating-point numbers may not represent every decimal fraction exactly.
- Repeated floating-point additions can introduce small rounding errors.
- Integer paise stores money as whole-number units and avoids fractional paise.
- An int can overflow when the total exceeds its maximum limit.
- A long long provides a much larger integer range.
-

7. Alternative Concept – Comparing Data Types

Data Type	Behaviour at 3,000,000 Journeys	Time per Addition	Space
int	Overflows	O(1)	4 bytes
long long	Stores 9 billion safely	O(1)	8 bytes

double	Stores the total with possible precision limitations for fractional values	O(1)	8 bytes

Evaluation:

- int is unsuitable for large network totals because of overflow.
- long long is suitable for storing integer-paise totals.
- double is useful for approximate decimal calculations but may introduce precision errors.
- Integer paise with long long is the preferred approach for fare calculations.

8. Result

Thus, floating-point and integer-paise totals were compared for 20,000, 500,000 and 3,000,000 journeys. The comparison demonstrated that int can overflow when storing large network totals, while long long safely stores the integer-paise total. Integer paise is therefore more reliable for financial calculations.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the final design for the metro.

Discuss:

• The types, constants and classes you recommend and why
• How a fare revision will be carried out
• The limitations of your design
• Whether the design remains correct for the whole network

Your recommendation must be based on evidence from your own experiments. Alternative Concept and Complexity Analysis

If the fare table were read from a configuration file when the gate starts, instead of being compiled into the program, what would the solution be? Analyse the extra start-up time to read z fare values and the space they occupy, against the cost of recompiling and redeploying after every revision.

Evidence Expected

A technically justified recommendation supported by your tests and measurements.

ANS:

1. Recommended Final Design

For the metro fare management system, I recommend an object-oriented design using a SmartCard class, integer paise for fare calculations, and separate functions for peak-hour checking and fare calculation.

This design provides accuracy, security and easier maintenance.

2. Recommended Data Types, Constants and Classes

Data Types:

- int: Used for zone numbers and hour values.
- long long: Used for balance and total fare in paise.
- bool: Used to identify peak and off-peak hours.
- enum class Zone: Used to represent Inner, Middle and Outer zones.

Constants:

```
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
```

```
constexpr long long BASE_PAISE[] = {
   1000, 2000, 3000
};
```
Constants prevent accidental changes to important fare rules.
SmartCard Class:
The SmartCard class should contain:

- Private balance data member.
- canTravel() function to check the remaining balance.
- deduct() function to deduct the fare safely.
- getBalance() function to display the balance.

This provides encapsulation and prevents direct modification of the balance.

3. How Fare Revision Will Be Carried Out

If the metro authority changes the fare:

1. Update the base fare constants.
2. Update the peak surcharge percentage if required.
3. Compile the updated program.
4. Test the revised fare calculations.
5. Deploy the updated program to the metro gates.

For example, if the Inner Zone fare changes from Rs. 10 to Rs. 12, update its base fare from 1000 to 1200 paise.

4. Limitations of the Design

- Fare changes require recompilation and redeployment.
- The current design supports only three zones.
- It does not include online payment or recharge integration.
- It does not store journey history permanently.
- It does not handle multiple metro stations and complex route combinations.

5. Correctness for the Whole Network

The design is suitable for the whole network when all gates use the same updated fare rules.
Using long long for network totals avoids the integer overflow observed with ordinary 32-bit int for large journey counts.
Integer paise also provides accurate money calculations without floating-point rounding errors.
However, large-scale deployment requires proper testing, centralized fare updates and consistent configuration across all gates.

6. Alternative Approach – Configuration File

Instead of compiling fares into the program, the metro can store fare details in a configuration file.
Example: fares.txt

```
INNER 1000
MIDDLE 2000
OUTER 3000
PEAK_SURCHARGE 20
MIN_BALANCE 5000
```

The program reads these values when the gate starts.

Example C++ code:

```
#include <iostream>
```

```
#include <fstream>
using namespace std;

int main()
{
   ifstream file("fares.txt");

   if (!file)
   {
      cout << "Unable to open fare configuration!";
      return 1;
   }

   string zone;
   long long fare;

   while (file >> zone >> fare)
   {
      cout << zone << " Fare: "
           << fare << " paise" << endl;
   }

   file.close();

   return 0;
}
```

7. Complexity Analysis

Let z represent the number of zones.

Feature	Compiled Fare Table	Configuration File
Start-up time	No file-reading overhead	O(z)
Fare lookup	O(1)	O(1) with suitable in-memory storage
Extra storage	O(z) for fare table	O(z) for loaded fare table
Fare revision	Requires recompilation	Edit configuration file
Deployment	Updated program required	Updated configuration required

Advantages of Configuration Files:

- Fare changes can be made without recompiling the program.
- The same program can support different fare structures.
- Centralized fare configuration makes maintenance easier.

Disadvantages:

- The configuration file must be available and valid.
- Incorrect values may cause incorrect fare calculations.
- File-reading errors must be handled safely.

8. Final Recommendation

I recommend an object-oriented design using a SmartCard class, long long integer-paise calculations, enum class for zones and centralized fare configuration.

The experiments in Tasks 6–10 demonstrated that:

- Switch statements correctly select zone fares.
- Boundary testing correctly identifies peak and off-peak hours.

- The minimum balance rule prevents invalid journeys.
- Integer paise avoids floating-point money-rounding issues.
- long long safely stores large network totals.

For a small prototype, compiled constants are sufficient. For a large metro network with frequent fare revisions, a validated configuration file is more flexible and easier to maintain.

9. Result

Thus, an object-oriented fare management system with integer-paise calculations and configurable fare rules is recommended. It provides accuracy, data protection, maintainability and scalability for metro fare processing.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The metro adds a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones.

Re-evaluate your original design. You may RETAIN → MODIFY → or REDESIGN your original solution. A. Impact Analysis – 2 Marks

Identify which constants, enum values, expressions, switch statements and classes are affected. Evidence Expected

A clear analysis of how the changed requirements affect the original design. B. Evaluation of the Original Design – 2 Marks

Evaluate whether the switch-based fare logic and the SmartCard class can absorb the changes, using worked examples.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe and implement the revised design (for example: a fare table indexed by zone, a separate discount rule, a Pass class).

Evidence Expected

An appropriate design response with working code.

D. Evidence-Based Defence – 2 Marks

Defend your decision using test results and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating tests and the changed requirements.

ANS:

A. Impact Analysis

The new requirements introduce a fourth zone, a weekend discount and a monthly pass. Therefore, the original fare system needs modifications.

1. Constants affected

The original base fare array must include the Airport Zone fare.

```
constexpr long long BASE_PAISE[] = {
    1000, 2000, 3000, 6000
};
```

```
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
constexpr long long MIN_BALANCE_PAISE = 5000;
```

2. Enum values affected

The Zone enum must include the new Airport zone.

```
enum class Zone {
   Inner, Middle, Outer, Airport
};
```

3. Expressions affected

The fare calculation must now consider three conditions:

- Normal fare.
- 20% peak surcharge.
- 10% weekend discount.

The weekend discount and peak surcharge must not be applied together.

4. Switch statements affected

The original switch statement must include an additional Airport case.

5. Classes affected

- SmartCard class must continue managing the balance.
- A new Pass class must manage monthly passes.
- The pass must store two selected zones and allow unlimited journeys in those zones during its validity period.

B. Evaluation of the Original Design

The original switch-based fare logic can support the fourth zone by adding another case. However, frequent changes to the number of zones require modifying the switch statement each time.

The original SmartCard class can manage balance deductions, but it needs modification to support monthly passes.

Worked Examples:

Zone	Normal Fare	Peak Fare	Weekend Fare
Inner	Rs. 10	Rs. 12	Rs. 9
Middle	Rs. 20	Rs. 24	Rs. 18
Outer	Rs. 30	Rs. 36	Rs. 27
Airport	Rs. 60	Rs. 72	Rs. 54

Example 1 – Airport Peak Journey

Base fare = Rs. 60

Peak surcharge = 20%

Final fare = Rs. 60 + Rs. 12 = Rs. 72

Example 2 – Airport Weekend Journey

Base fare = Rs. 60

Weekend discount = 10%

Final fare = Rs. 60 − Rs. 6 = Rs. 54

Example 3 – Monthly Pass

If a passenger purchases a monthly pass for Inner and Middle zones, unlimited journeys are allowed in those two zones during the pass validity period.

No additional journey fare is deducted for journeys within the selected zones.
Evaluation: The original design is partially suitable, but it requires modifications to support the new requirements.

C. Adaptation or Redesign

I recommend modifying the original design by using:

1. An array-based fare table.
2. Separate peak surcharge and weekend discount conditions.
3. A SmartCard class for balance management.
4. A Pass class for monthly pass management.

Complete Revised C++ Program

```
#include <iostream>
using namespace std;

enum class Zone {
    Inner, Middle, Outer, Airport
};

constexpr long long BASE_PAISE[] = {
    1000, 2000, 3000, 6000
};

constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
constexpr long long MIN_BALANCE_PAISE = 5000;

class Pass {
private:
    Zone zone1, zone2;
    bool active;

public:
    Pass(Zone z1, Zone z2) {
        zone1 = z1;
        zone2 = z2;
        active = true;
    }

    bool allows(Zone z) {
        return active && (z == zone1 || z == zone2);
    }
};

class SmartCard {
private:
    long long balance;

public:
    SmartCard(long long initialBalance) {
        balance = initialBalance;
    }

    bool canTravel(long long fare) {
        return balance - fare >= MIN_BALANCE_PAISE;
    }

    void deduct(long long fare) {
        if (canTravel(fare)) {
```

```
            balance -= fare;
            cout << "Fare deducted: Rs. "
                 << fare / 100.0 << endl;
        }
        else {
            cout << "Travel denied: Low balance\n";
        }
    }

    long long getBalance() {
        return balance;
    }
};

long long calculateFare(
    Zone z, bool peak, bool weekend)
{
    long long fare =
        BASE_PAISE[static_cast<int>(z)];

    if (weekend) {
        fare -= fare * WEEKEND_DISCOUNT_PERCENT / 100;
    }
    else if (peak) {
        fare += fare * PEAK_SURCHARGE_PERCENT / 100;
    }

    return fare;
}

int main() {

    SmartCard card(20000);

    Pass monthlyPass(Zone::Inner, Zone::Middle);

    Zone journeys[] = {
        Zone::Inner,
        Zone::Airport,
        Zone::Outer,
        Zone::Middle
    };

    bool peak[] = {
        false, true, false, true
    };

    bool weekend[] = {
        false, false, true, false
    };

    for (int i = 0; i < 4; i++) {

        Zone z = journeys[i];

        cout << "\nJourney " << i + 1 << endl;

        if (monthlyPass.allows(z)) {
            cout << "Monthly pass accepted. No fare deducted.\n";
        }
        else {
```

```
            long long fare =
                calculateFare(z, peak[i], weekend[i]);

            card.deduct(fare);
        }

        cout << "Remaining balance: Rs. "
             << card.getBalance() / 100.0 << endl;
    }

    return 0;
}
```

Expected Output

Journey 1
Monthly pass accepted. No fare deducted.
Remaining balance: Rs. 200

Journey 2
Fare deducted: Rs. 72
Remaining balance: Rs. 128

Journey 3
Fare deducted: Rs. 27
Remaining balance: Rs. 101

Journey 4
Monthly pass accepted. No fare deducted.
Remaining balance: Rs. 101

D. Evidence-Based Defence

The modified design is more suitable for the updated metro requirements.
Evidence from testing:

Test Case	Expected Result	Actual Result
Inner normal journey	Rs. 10	Rs. 10
Airport peak journey	Rs. 72	Rs. 72
Outer weekend journey	Rs. 27	Rs. 27
Inner monthly-pass journey	Free	Free
Middle monthly-pass journey	Free	Free

Advantages:

- The array-based fare table makes adding new zones easier.
- Separate discount conditions prevent combining weekend discounts with peak surcharges.
- The SmartCard class protects the balance through private data.
- The Pass class supports unlimited journeys in two selected zones.

Limitations:

- Monthly pass expiry dates are not implemented in this prototype.
- Pass purchase and renewal are not included.

- Only two zones are supported per pass.
- The program does not store journey history.

Complexity Analysis

Operation	Time Complexity	Space Complexity
Fare lookup	O(1)	O(1)
Fare calculation	O(1)	O(1)
Pass zone checking	O(1)	O(1)
Journey processing	O(n)	O(n) for journey arrays

Final Recommendation

I recommend modifying the original design rather than completely redesigning it. The array-based fare table, separate discount logic, SmartCard class and new Pass class provide a flexible solution for the updated metro system.

The test results demonstrate that the revised system correctly calculates Airport fares, applies weekend discounts instead of peak surcharges, and accepts journeys covered by monthly passes.

Result

Thus, the original metro fare system was successfully adapted to support four zones, weekend discounts and monthly passes. The revised object-oriented design improves flexibility, maintainability and fare management while preserving the original balance protection mechanism.